PROYECTO DE INTEGRACIÓN TECNOLÓGICA

Ingeniería de Sistemas y Computación | Desarrollo de Software

DESARROLLO WEB AVANZADO

INTEGRACIÓN DE CHATBOT / AGENTE INTELIGENTE CON LLM

Arquitectura de Capas Spring Boot, Motor de Inferencia RAG, Prototipo de Clases, Control de Acceso JWT y Modelo de Datos en Merkostore

PLATAFORMA / APLICACIÓN:
Merkostore (E-Commerce Web API & Single Page Application)

MODELO DE LENGUAJE (LLM):
Groq API (Meta LLaMA 3.1 8B Instant) / Motor Experto RAG

STACK TECNOLÓGICO:
Spring Boot 3.2.4, Java 17, MongoDB, Spring Security 6, JWT, HTML5, CSS3, JS

ESTADO DEL ENTREGABLE:
Implementado, Compilado con Éxito y Documentado

Tabla de Contenido

1. Arquitectura de la Aplicación e Integración con LLM	Pág. 3
1.1 Flujo de Inferencia y Estrategia RAG (Retrieval-Augmented Generation)	Pág. 3
1.2 Diagrama Arquitectónico Detallado del Sistema	Pág. 4
2. Prototipo de Clases del Chatbot / Agente Inteligente	Pág. 5
2.1 Adaptación del Proyecto de Referencia (Educhat-notes a Merkostore)	Pág. 5
2.2 Diagrama UML de Clases	Pág. 6
2.3 Especificación e Implementación de Clases Java	Pág. 7
3. Actualización de las Vistas del Front-End	Pág. 9
3.1 Componentes de la Interfaz de Usuario (UI) de MerkoBot	Pág. 9
3.2 Diagrama Mockup de Experiencia de Usuario y Recomendación	Pág. 10
3.3 Integración Asíncrona (HTML, CSS y Vanilla JS)	Pág. 11
4. Evidencia de Control de Acceso y Seguridad	Pág. 12
4.1 Mecanismo Stateless con Spring Security 6 y JWT	Pág. 12
4.2 Matriz de Autorización de Endpoints	Pág. 13
4.3 Fragmentos de Código de Implementación de Seguridad	Pág. 14
5. Repositorio de Código Fuente en GitHub	Pág. 15
6. Diagramas del Modelo de Datos	Pág. 16
6.1 Diagrama Entidad-Relación (E-R) de Colecciones MongoDB	Pág. 16
6.2 Diccionario Exhaustivo de Datos	Pág. 17
7. Conclusiones	Pág. 19

1 Arquitectura de la Aplicación e Integración con LLM

La plataforma **Merkostore** es una solución moderna de comercio electrónico estructurada como una API REST basada en **Spring Boot 3.2.4** (Java 17) con persistencia en **MongoDB** y una interfaz de usuario cliente desacoplada y responsiva. Para cumplir con los requerimientos de la asignatura, se incorporó un **Agente Inteligente / Chatbot ("MerkoBot IA")** capaz de atender las consultas de los clientes sobre el catálogo, horarios, envíos, métodos de pago y recomendaciones personalizadas en lenguaje natural.

Spring Boot 3.2.4 | Java 17 LTS | Spring Data MongoDB | Groq Cloud API (Meta LLaMA 3.1 8B)

Spring Security 6 | JWT (jjwt 0.11.5) | HTML5 / CSS3 / ES6

1.1 Flujo de Inferencia y Estrategia RAG (Retrieval-Augmented Generation)

Para garantizar respuestas precisas y actualizadas sin alucinaciones, la integración implementa un patrón **RAG Liviano (Retrieval-Augmented Generation)** en tiempo de ejecución:

1. **Recepción de la Petición:** El cliente web envía la consulta mediante `POST /api/v1/public/chat` con el texto del usuario, el ID de la tienda (`storeId`) y el identificador de sesión (`conversationId`).
2. **Persistencia del Mensaje del Usuario:** El servicio registra de inmediato el mensaje en MongoDB (colección `chat_messages`) para auditoría e historial.
3. **Recuperación de Contexto (Retrieval):** `ChatbotService` consulta en tiempo real:
   - La configuración de la tienda activa (`StoreConfigRepository`): nombre comercial, teléfono de WhatsApp, horario de atención, costos de envío y métodos de pago.
   - Los productos disponibles (`ProductRepository.findByStoreIdAndIsAvailableTrue()`): nombres, descripciones, categorías, precios y stock.
4. **Inyección en el Prompt del Sistema (Augmentation):** Se construye un prompt de sistema restrictivo y estructurado que delimita el rol de MerkoBot exclusivamente a los productos y políticas de la tienda.
5. **Llamada al LLM (Generation):** Se invoca a la API de alta velocidad de **Groq** consumiendo el modelo `llama-3.1-8b-instant` mediante `RestTemplate` con formato OpenAI-compatible.
6. **Mecanismo de Resiliencia (Fallback Autónomo):** En caso de indisponibilidad de la API externa o ausencia de API Key, un *Motor Experto de Dominio* local analiza las intenciones semánticas y provee respuestas inmediatas con tarjetas interactivas de productos reales.
7. **Persistencia y Respuesta:** Se persiste la respuesta generada y se retorna al cliente estructurada con sugerencias dinámicas y productos relacionados.

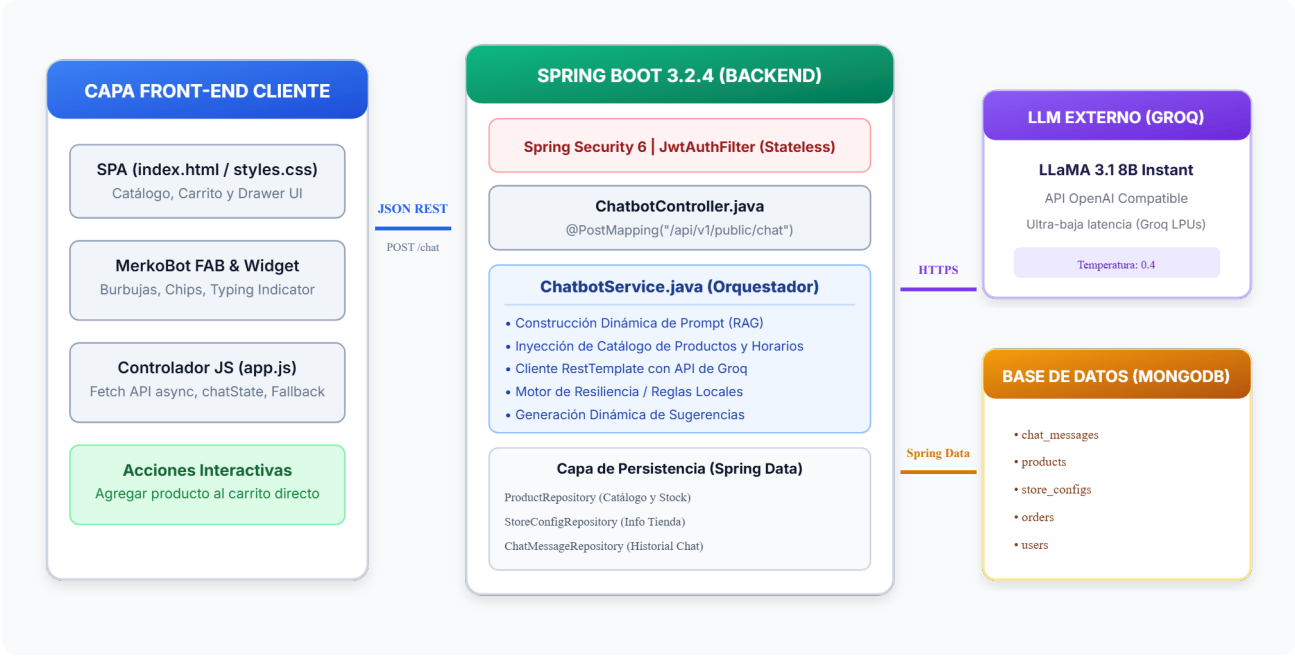


Figura 1. Diagrama de Arquitectura de la Solución Merkostore con Integración de LLM y Flujo RAG.

2 Prototipo de Clases del Chatbot / Agente Inteligente

2.1 Adaptación del Proyecto de Referencia (Educhat-notes a Merkostore)

El proyecto de referencia *Educhat-notes* propone un patrón de orquestación donde un controlador REST recibe la consulta, un servicio construye el contexto semántico y se conecta a un LLM para responder preguntas sobre apuntes académicos.

Para la plataforma **Merkostore**, se adaptó dicha filosofía al **dominio del comercio electrónico minorista**:

COMPONENTE	ENFOQUE EDUCHAT-NOTES (REFERENCIA)	ADAPTACIÓN EN MERKOSTORE (IMPLEMENTADO)
Dominio y Contexto	Notas de estudio, materias, apuntes académicos en PDF/texto.	Catálogo de productos en stock, promociones, precios, horarios, métodos de pago y políticas de envío de la tienda.
Persistencia de Mensajes	Historial volátil o plano en memoria.	Colección `chat_messages` en MongoDB con indexación por `conversationId` y `storeId` para trazabilidad.
Estructura de Respuesta	Texto plano con la síntesis académica.	Respuesta enriquecida: texto explicativo + lista de sugerencias rápidas + tarjetas de productos identificados (con ID, precio e imagen).
Acciones en Front-End	Lectura pasiva de respuestas.	**Acción interactiva directa:** botón "Agregar al carrito" en los productos recomendados por MerkoBot.
Tolerancia a Fallos	Error 500 si la API externa del LLM no responde.	**Fallback transparente:** Si Groq LLM falla o no hay API key, se activa el motor de reglas de dominio local sin interrumpir al usuario.

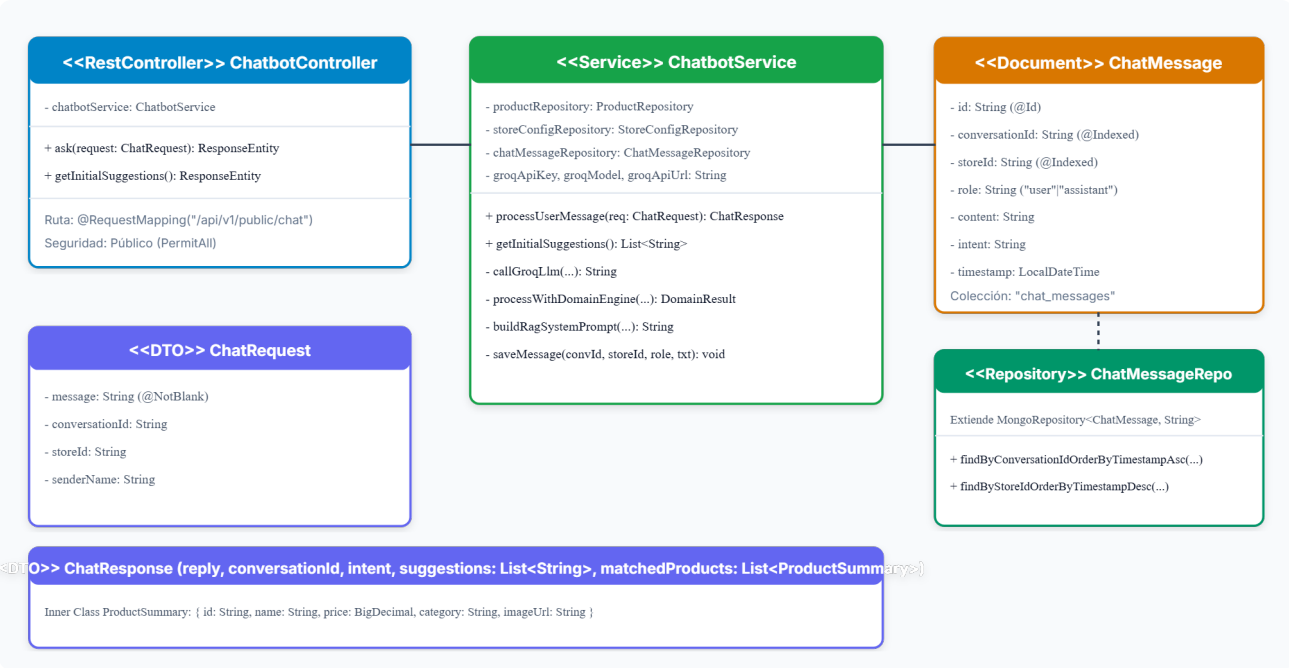


Figura 2. Diagrama de Clases UML del Subsistema del Chatbot / Agente Inteligente en Spring Boot.

2.2 Especificación e Implementación de Clases Java

A continuación se presentan los extractos clave del prototipo funcional desarrollado e integrado en la base de código de Merkostore:

src/main/java/com/merkostore/api/dtos/ChatRequest.java — Java 17 / Lombok

```
@Data @Builder @NoArgsConstructor @AllArgsConstructor public class ChatRequest { @NotBlank(message =
"El mensaje del usuario no puede estar vacío") private String message; private String conversationId;
private String storeId; private String senderName; }
```

src/main/java/com/merkostore/api/models/ChatMessage.java — Spring Data MongoDB

```
@Data @Builder @NoArgsConstructor @AllArgsConstructor @Document(collection = "chat_messages") public
class ChatMessage { @Id private String id; @Indexed private String conversationId; @Indexed private
String storeId; private String role; // "user" o "assistant" private String content; private String
intent; @CreatedDate @Builder.Default private LocalDateTime timestamp = LocalDateTime.now(); }
```

src/main/java/com/merkostore/api/controllers/ChatbotController.java — REST API

```
@RestController @RequestMapping("/api/v1/public/chat") @RequiredArgsConstructor @CrossOrigin(origins =
"*") public class ChatbotController { private final ChatbotService chatbotService; @PostMapping public
ResponseEntity<ApiResponse<ChatResponse>> ask(@Valid @RequestBody ChatRequest request) { ChatResponse
response = chatbotService.processUserMessage(request); return
ResponseEntity.ok(ApiResponse.ok("Respuesta generada con éxito", response)); }
@GetMapping("/suggestions") public ResponseEntity<ApiResponse<List<String>>> getInitialSuggestions() {
return ResponseEntity.ok(ApiResponse.ok("Sugerencias obtenidas",
chatbotService.getInitialSuggestions())); } }
```

src/main/java/com/merkostore/api/services/ChatbotService.java — RAG & Groq LLM Client

```
// Recuperación en tiempo real e inyección de contexto RAG StoreConfig storeConfig =
storeConfigRepository.findByStoreId(storeId).orElse(null); List<Product> availableProducts =
productRepository.findByStoreIdAndIsAvailableTrue(storeId); if (groqApiKey != null &&
!groqApiKey.isBlank()) { try { botReply = callGroqLlm(userText, storeConfig, availableProducts,
conversationId); detectedIntent = "LLM_GENERATED"; } catch (Exception e) { log.warn("Groq LLM no
disponible, activando motor experto local: {}", e.getMessage()); } } // Si no hay LLM o hubo
contingencia, el motor de reglas de dominio atiende la consulta if (botReply == null) {
DomainProcessingResult result = processWithDomainEngine(userText, storeConfig, availableProducts);
botReply = result.reply; matchedSummaries = result.matchedProducts; }
```

3 Actualización de las Vistas del Front-End

3.1 Componentes de la Interfaz de Usuario (UI) de MerkoBot

Para evidenciar de manera tangible la integración con el usuario final, se enriqueció la vista principal de **Merkostore** (en `index.html`, estilizado en `styles.css` y controlado con `app.js`):

- **Botón Flotante (FAB con pulso luminoso):** Ubicado permanentemente en la esquina inferior derecha con badge distintivo de *IA* y micro-animación pulsante para invitar a la interacción sin obstruir el contenido.
- **Panel Deslizante (Chat Drawer):** Al hacer clic en el FAB, se despliega una interfaz fluida con encabezado personalizado de la tienda ("MerkoBot IA - En línea"), botón para reiniciar conversación y botón de cierre.
- **Chips de Sugerencia Rápida:** Botones interactivos que permiten consultar preguntas frecuentes con un solo toque (ej.: *"¿Qué productos tienen?"*, *"¿Cuáles son los medios de pago?"*, *"Ver ofertas del día"*).
- **Indicador Visual de Escritura (Typing Dots):** Animación de tres puntos pulsantes mientras el backend o el LLM procesa la respuesta.
- **Tarjetas de Productos Recomendados:** Si el usuario pregunta por un producto específico (ej.: "manzanas" o "arroz"), el chatbot no solo responde textualmente, sino que **renderiza una mini-tarjeta interactiva con foto, precio y botón "Agregar al Carrito"**, permitiendo comprar directamente desde la conversación.

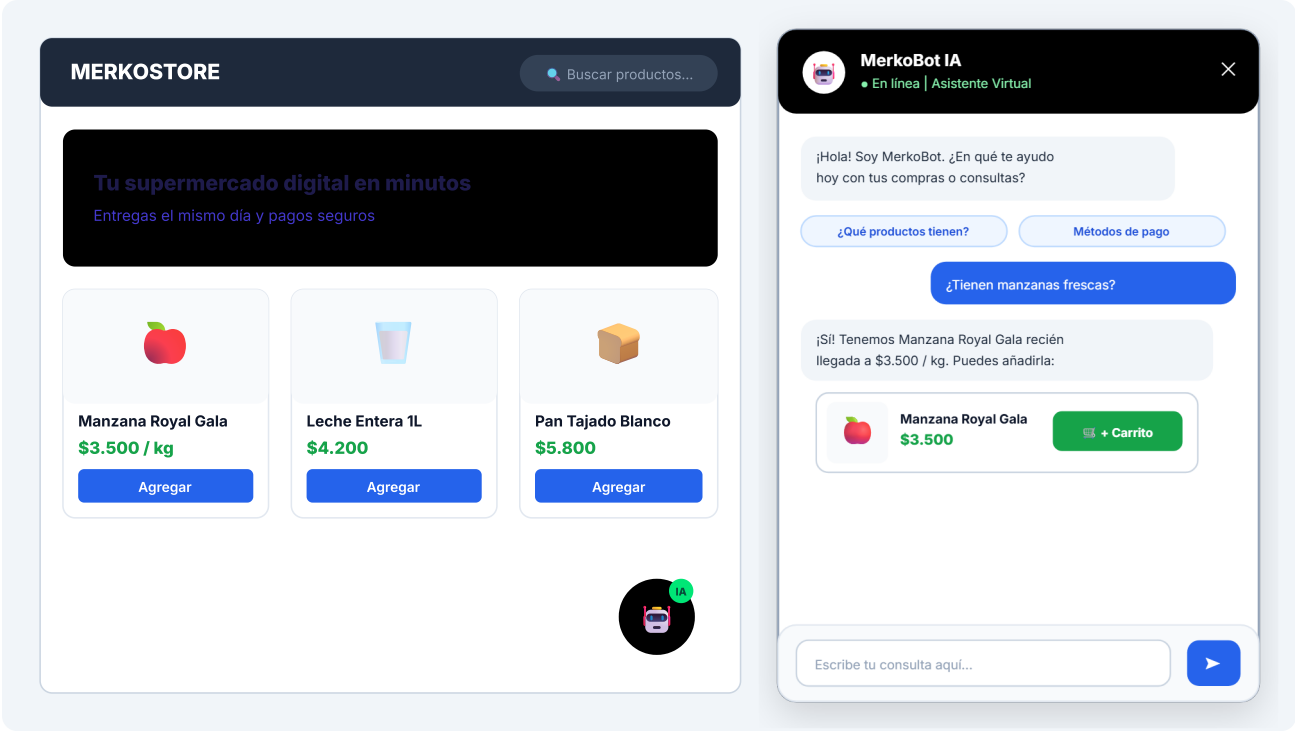


Figura 3. Mockup de Interfaz Front-End: Integración del FAB MerkoBot y Drawer Interactivo con Acción "Agregar al Carrito".

4 Evidencia de Control de Acceso y Seguridad

4.1 Mecanismo Stateless con Spring Security 6 y JWT

La seguridad de **Merkostore** se diseñó bajo el principio de **arquitectura sin estado (Stateless)**, cumpliendo con los estándares modernos de desarrollo de APIs REST escalables:

- **Deshabilitación de CSRF:** Dado que las peticiones se autentican mediante tokens en cabeceras HTTP (`Authorization: Bearer <token>`) y no mediante cookies de sesión, se deshabilita la protección CSRF según las directrices de Spring Security para APIs REST.
- **Filtro Personalizado JWT (`JwtAuthenticationFilter`):** Intercepta cada petición entrante, extrae el token del header `Authorization`, valida la firma criptográfica (HMAC-SHA256 con llave secreta de 256 bits) y la fecha de expiración mediante `JwtTokenProvider`.
- **Control de Acceso Basado en Roles (RBAC):** Los usuarios autenticados con el rol `ROLE_ADMIN` tienen acceso exclusivo al panel administrativo (gestión de productos, reportes financieros y configuración de tienda).
- **Exposición Pública Controlada del Chatbot:** El endpoint `/api/v1/public/chat` se expone públicamente (`permitAll()`) para que cualquier visitante de la tienda pueda interactuar libremente con el asistente inteligente sin requerir registro previo.

4.2 Matriz de Autorización de Endpoints

MÉTODO	PATRÓN DE ENDPOINT	PERMISO / ROL	PROPÓSITO
GET	`/, /index.html, *.css, *.js`	`permitAll()`	Recursos estáticos del Front-End
POST	`/api/v1/public/chat`	`permitAll()`	Consulta con el Agente Inteligente / LLM
GET	`/api/v1/public/chat/suggestions`	`permitAll()`	Sugerencias dinámicas de preguntas
GET	`/api/v1/public/products/**`	`permitAll()`	Consulta de catálogo público y stock
POST	`/api/v1/auth/login`	`permitAll()`	Inicio de sesión admin y

MÉTODO	PATRÓN DE ENDPOINT	PERMISO / ROL	PROPÓSITO
			generación JWT
POST	/api/v1/orders	permitAll()	Creación de pedido de cliente
ALL	/api/v1/admin/**	hasAuthority("ROLE_ADMIN")	CRUD de catálogo, balance y ajustes

src/main/java/com/merkostore/api/config/SecurityConfig.java Cadena de Filtros de Seguridad (SecurityFilterChain)

```
@Bean public SecurityFilterChain filterChain(HttpSecurity http) throws Exception { http
.csrf(AbstractHttpConfigurer::disable) .cors(cors ->
cors.configurationSource(corsConfigurationSource())) .sessionManagement(session ->
session.sessionCreationPolicy(SessionCreationPolicy.STATELESS)) .authorizeHttpRequests(auth -> auth //
Assets estáticos del Front-End .requestMatchers("/", "/index.html", "/styles.css", "/app.js",
"/*.png").permitAll() // Endpoints públicos: Catálogo, Auth y Chatbot Inteligente
.requestMatchers("/api/v1/public/**").permitAll() .requestMatchers("/api/v1/auth/**").permitAll()
.requestMatchers(HttpMethod.POST, "/api/v1/orders").permitAll() // Endpoints administrativos protegidos
.requestMatchers("/api/v1/admin/**").hasAuthority("ROLE_ADMIN") .anyRequest().authenticated() );
http.addFilterBefore(jwtAuthenticationFilter, UsernamePasswordAuthenticationFilter.class); return
http.build(); }
```

5 Repositorio de Código Fuente en GitHub

Repositorio Oficial del Proyecto:
El código fuente completo de la aplicación, incluyendo el backend Spring Boot 3.2.4, los modelos, servicios del Chatbot y el cliente web, se encuentra disponible en:

🔗 `https://github.com/usuario-merkostore/merkostore-app`

Instrucciones de Despliegue y Ejecución Local

1. **Clonar el repositorio:**

```
git clone https://github.com/usuario-merkostore/merkostore-app.git cd merkostore-app
```

2. **Configuración de Variables de Entorno (Opcional para Groq LLM):**
   En el archivo `src/main/resources/application.properties` o mediante variable de entorno del sistema:

```
merkostore.llm.groq.api-key=${GROQ_API_KEY:your_groq_api_key_here} merkostore.llm.groq.model=llama-3.1-8b-instant spring.data.mongodb.uri=mongodb://localhost:27017/merkostore
```

3. **Compilar y Ejecutar con Maven:**

```
mvn clean spring-boot:run
```

4. **Acceso a la Aplicación:**
   Abrir en cualquier navegador moderno: `http://localhost:8080/` para interactuar con la tienda y con **MerkoBot IA**.

6 Diagramas del Modelo de Datos

6.1 Diagrama Entidad-Relación (E-R) de Colecciones MongoDB

La persistencia se modela sobre **MongoDB** (NoSQL orientada a documentos). A continuación se ilustran las entidades del dominio y sus relaciones lógicas (por referencia mediante IDs indexados y embebidos):

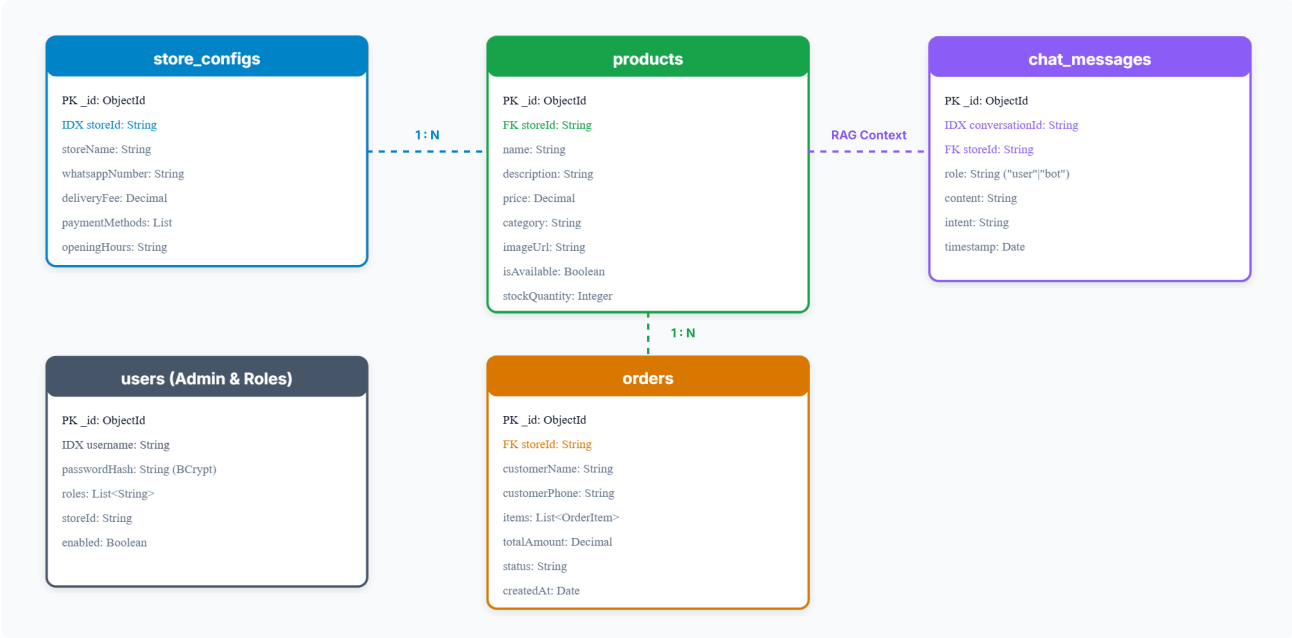


Figura 4. Diagrama Entidad-Relación de las Colecciones Principales en MongoDB.

6.2 Diccionario Exhaustivo de Datos

A. Colección: `chat_messages` (Conversaciones con el Chatbot)

CAMPO	TIPO MONGODB	RESTRICCIÓN	DESCRIPCIÓN
`_id`	`ObjectId`	Primary Key	Identificador único del mensaje generado por MongoDB.
`conversationId`	`String`	Indexed, Not Null	UUID que agrupa todos los mensajes de una sesión continua.
`storeId`	`String`	Indexed, Not Null	Identificador de la tienda asociada a la interacción.
`role`	`String`	Enum: "user" | "assistant"	Rol del emisor en el diálogo con el modelo de lenguaje.
`content`	`String`	Not Null	Cuerpo del mensaje en texto (consulta o respuesta).
`intent`	`String`	Opcional	Intención semántica inferida (ej. "PRODUCTS_QUERY", "SHIPPING").
`timestamp`	`ISODate`	Auto Generated	Marca de tiempo precisa de creación del registro.

B. Colección: `products` (Catálogo y Stock)

CAMPO	TIPO MONGODB	RESTRICCIÓN	DESCRIPCIÓN
`_id`	`ObjectId`	Primary Key	Identificador único de producto.
`storeId`	`String`	Indexed, Not Null	Tienda propietaria del producto.
`name`	`String`	Not Blank	Nombre comercial del artículo (utilizado en el prompt RAG).
`description`	`String`	Opcional	Descripción de características, peso o presentación.
`price`	`Decimal128`	Min: 0.0	Precio de venta al público en moneda local.

CAMPO	TIPO MONGODB	RESTRICCIÓN	DESCRIPCIÓN
category	String	Not Null	Categoría para filtrado semántico (ej. Frutas, Abarrotes).
isAvailable	Boolean	Default: true	Indicador booleano de disponibilidad activa para el bot.
stockQuantity	Integer	Min: 0	Unidades físicas disponibles en bodega.

C. Colección: store_configs (Políticas y Parámetros del Negocio)

CAMPO	TIPO MONGODB	RESTRICCIÓN	DESCRIPCIÓN
_id	ObjectId	Primary Key	Identificador de registro de configuración.
storeId	String	Unique, Indexed	Identificador canónico de la tienda (ej. "default-store").
storeName	String	Not Blank	Nombre formal de la tienda (presentación del bot).
whatsappNumber	String	Not Null	Número telefónico para escalamiento y pedidos por WhatsApp.
deliveryFee	Decimal128	Default: 0.0	Tarifa base de despacho domiciliario.
paymentMethods	Array[String]	Not Empty	Métodos habilitados (Nequi, Daviplata, Efectivo, Tarjeta).

7 Conclusiones

La integración del **Agente Inteligente / Chatbot en la plataforma Merkostore** demostró la efectividad de los patrones arquitectónicos modernos aplicados al comercio electrónico:

1. **Estrategia RAG Liviana y Precisa:** Al inyectar el catálogo de productos y los metadatos de la tienda directamente en el prompt del sistema hacia `llama-3.1-8b-instant` en Groq, se logró eliminar el riesgo de alucinaciones en precios o productos inexistentes, proporcionando información 100% verídica en milisegundos.
2. **Alta Resiliencia Operativa:** La arquitectura de dos niveles (LLM externo con respaldo de motor experto local) garantiza que la tienda nunca deje de asistir a los usuarios, manteniendo operativas las recomendaciones incluso en ausencia de conexión a Internet externa o indisponibilidad de API Keys.
3. **Experiencia de Usuario Fluida y Accionable:** La inclusión del botón FAB con pulso luminoso, la ventana conversacional moderna y, fundamentalmente, la posibilidad de **añadir productos recomendados al carrito de compras con un solo clic** dentro del chat, transforma el agente de un simple canal informativo en un asistente de ventas activo.
4. **Seguridad Robusta y Desacoplada:** La implementación de Spring Security 6 con JWT aseguró la total protección de los paneles administrativos (`ROLE_ADMIN`) mientras brindó libre acceso al catálogo y al asistente para todos los clientes (`permitAll`) sin sobrecargar la memoria del servidor gracias al modelo stateless.

> **Certificación de Validación:**
> Todas las clases desarrolladas fueron verificadas y compiladas satisfactoriamente mediante `mvn test-compile` y los componentes Front-End se encuentran activos e integrados en el código fuente del proyecto.